

PROPLEDGER ARCHITECTURE SERIES — VOLUME 02 OF 10

Relational Schema & Modeling

PostgreSQL 16 Schema Blueprint, Constraints & Entity Relationships

DOMAIN LEVEL:

Enterprise Real Estate SaaS

Comparable System: Yardi Voyager / RealPage

FRAMEWORKS:

Spring Boot 3.3 | PG 16 | React 19

PAGE BUDGET:

10 Pages Complete Reference

1. DOCUMENT EXECUTIVE ABSTRACT & SCOPE

This volume presents the comprehensive relational database schema and data architecture of PropLedge. Designed to institutional property management standards, it details the 19 core relational entities, check constraints, foreign key cascades, JSONB audit structures, and temporal exclusion models that guarantee zero financial corruption and mathematical occupancy integrity.

2. VOLUME SYLLABUS & CORE CAPABILITIES

Page	Chapter Title	Primary Technical Focus & Learning Outcome
02	Relational Architecture Philosophy	Design principles, financial immutability, and 3NF discipline
03	Asset Hierarchy Model	Properties, buildings, units, and structural cardinality rules
04	Resident & Tenant Domain	Tenants, co-signers, roommates, guarantors, and emergency contacts
05	Lease Contract Architecture	Temporal intervals, payment due dates, and btree_gist exclusions
06	Accounts Receivable (AR) Ledger	Invoices, line items, non-negative balance checks, and invariants
07	Payment Settlement Subledger	Double-entry payment_allocations, split distributions, and receipts
08	Accounts Payable & Facilities	Vendors, operating expenses, maintenance tickets, and work orders
09	Compliance & Audit Logging	Append-only audit_logs, JSONB old/new diffs, and SOX controls
10	Database Modeling Interview Q&A	Real-world schema trade-offs, UUID vs BIGSERIAL, and questions

SENIOR STAFF ENGINEER REVIEWER GUIDANCE

This volume is engineered specifically for software engineering and database architecture interviews at Tier-1 real estate technology and enterprise SaaS corporations (Yardi, RealPage, AppFolio, CoStar). Every code sample reflects active, compilable implementations from the PropLedge production repository.

2. RELATIONAL DATA ARCHITECTURE & DESIGN PHILOSOPHY

FOUNDATIONS OF FINANCIAL & TEMPORAL DATA INTEGRITY

1. Core Architectural Pillars

- In real estate asset management, software governs billions of dollars in rent rolls, security deposits, and vendor payouts. PropLedger establishes four immutable database design principles:
- 1. Relational Integrity Over Application Checks: While Spring Boot validates incoming DTOs, PostgreSQL strictly enforces business invariants via foreign keys, check constraints, and exclusion constraints. Even direct psql writes cannot corrupt data.
 - 2. Double-Entry Style Financial Ledger: Payments never directly mutate invoice totals. A dedicated junction settlement entity (payment_allocations) bridges cash receipts to receivables, preserving complete accounting auditability.
 - 3. Native Temporal Intervals: Lease dates are modeled as first-class temporal ranges (daterange) evaluated using PostgreSQL range algebra to prevent double-booking.
 - 4. Immutable Audit Ledger: State changes to sensitive entities are captured in an append-only audit trail with JSONB deltas, preventing repudiation.

2. Primary Key Strategy: Surrogate vs Natural Keys

PropLedger adopts 64-bit sequential BIGINT surrogate primary keys (BIGSERIAL) internally, paired with unique business codes for external exposure:

Key Type	SQL Representation	Primary PropLedger Use Case	Performance & Security Rationale
Surrogate PK	id BIGINT GENERATED ALWAYS	Internal entity identity & FK joins	Compact 8-byte B-Tree index, zero page fragmenta...
Business Code	lease_number VARCHAR(50) UK	User-facing contract references	Human-readable (e.g. LSE-2026-0042) with B-Tree...
UUID (v4)	public_id UUID DEFAULT gen_u...	Public REST API URL parameters	Prevents enumeration attacks across REST endpo...

3. Currency & Time Standards

Financial amounts are strictly defined as NUMERIC(14,2) (supporting up to \$999 billion with exact decimal precision). FLOAT and DOUBLE are barred to prevent IEEE 754 rounding drift. All timestamps are TIMESTAMPTZ (UTC).

3. PHYSICAL ASSET HIERARCHY: PROPERTIES, BUILDINGS & UNITS

MODELING COMPLEX REAL ESTATE PORTFOLIOS

1. The Three-Tier Asset Taxonomy

Commercial real estate assets range from single high-rises to multi-structure garden complexes. PropLedge models this flexibility with a 3-tier hierarchy:

- **Property Level (Asset):** The legal real estate asset or corporate entity (e.g. Grandview Heights). Holds `owner_id`, tax jurisdiction, and address.
- **Building Level (Structure):** Optional intermediate physical structure (e.g. Tower A, North Wing). Groups units by physical elevator or wing.
- **Unit Level (Rentable Space):** The rentable residential dwelling or commercial suite. Holds `market_rent`, bedrooms, bathrooms, and current operational status.

2. PostgreSQL DDL: Properties and Units

// CODE LISTING: SQL DDL – ASSET HIERARCHY IMPLEMENTATION

```
-- Properties Table (V2__create_properties.sql)
CREATE TABLE properties (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  owner_id BIGINT NOT NULL REFERENCES owners(id) ON DELETE RESTRICT,
  property_code VARCHAR(30) NOT NULL UNIQUE,
  name VARCHAR(150) NOT NULL,
  property_type VARCHAR(40) NOT NULL CHECK (property_type IN
    ('RESIDENTIAL_MULTIFAMILY', 'COMMERCIAL_OFFICE', 'MIXED_USE', 'INDUSTRIAL')),
  address_line1 VARCHAR(200) NOT NULL,
  city VARCHAR(100) NOT NULL, state VARCHAR(50) NOT NULL, postal_code VARCHAR(20) NOT NULL,
  year_built INTEGER CHECK (year_built >= 1800),
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Units Table with Composite Uniqueness (V3__create_buildings_units.sql)
CREATE TABLE units (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

3. Composite Uniqueness Guarantee

The constraint 'UNIQUE (property_id, building_id, unit_number)' ensures that duplicate unit numbers cannot be created within the same building, while allowing identical unit numbers across separate structures.

4. RESIDENT & TENANT DOMAIN: PROFILES & GUARANTORS

ONBOARDING, ROOMMATE RELATIONSHIPS & FAIR HOUSING COMPLIANCE

1. Modeling Multi-Tenant Occupancies

In residential operations, single units are frequently leased by multiple roommates, or require a parent guarantor who assumes financial liability without physical occupancy. PropLedge models this via the 'lease_tenants' junction table.

2. DDL: Tenants, Emergency Contacts & LeaseTenants

```
// CODE LISTING: SQL DDL – RESIDENT SUBSYSTEM SCHEMA
```

```
-- Tenants Table (V4__create_tenants.sql)
CREATE TABLE tenants (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id BIGINT UNIQUE REFERENCES users(id) ON DELETE SET NULL,
  current_lease_id BIGINT, -- Forward pointer to active lease
  first_name VARCHAR(80) NOT NULL,
  last_name VARCHAR(80) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  phone VARCHAR(30) NOT NULL,
  date_of_birth DATE NOT NULL,
  credit_score INTEGER CHECK (credit_score BETWEEN 300 AND 850),
  annual_income NUMERIC(14,2) CHECK (annual_income >= 0),
  employment_status VARCHAR(50) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Lease Tenants Junction Table (V5__create_leases.sql)
```

3. Fair Housing & PII Protection Guidelines

Credit scores and income figures are subject to Fair Housing Act regulations. Under PropLedge's security model, tenant PII is visible only to authorized Property Managers and Super Admins. If a tenant moves out, their record is preserved under statutory 7-year IRS tax rules.

5. LEASE CONTRACTS & TEMPORAL RANGE ARCHITECTURE

POSTGRESQL BTREE_GIST RANGE EXCLUSION ENGINEERING

1. The Temporal Double-Booking Concurrency Bug

In real estate software, a critical failure mode occurs when two agents lease the same unit for overlapping dates simultaneously. Standard SQL 'SELECT COUNT(*)...' checks fail under concurrent load because both transactions execute before either commits.

2. The GiST Exclusion Constraint Solution

PropLedger resolves this at the database engine level using PostgreSQL's 'btree_gist' extension and native 'daterange' types:

// CODE LISTING: SQL DDL – TEMPORAL EXCLUSION CONSTRAINT

```
-- V5__create_leases.sql
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE leases (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  lease_number VARCHAR(50) NOT NULL UNIQUE,
  unit_id BIGINT NOT NULL REFERENCES units(id) ON DELETE RESTRICT,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  rent_amount NUMERIC(12,2) NOT NULL CHECK (rent_amount > 0),
  deposit_amount NUMERIC(12,2) NOT NULL DEFAULT 0.00,
  payment_due_day INTEGER NOT NULL DEFAULT 1 CHECK (payment_due_day BETWEEN 1 AND 28),
  status VARCHAR(30) NOT NULL DEFAULT 'DRAFT'
  CHECK (status IN ('DRAFT', 'ACTIVE', 'TERMINATED', 'EXPIRED', 'RENEWED')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_lease_dates CHECK (end_date > start_date),

  -- TEMPORAL EXCLUSION CONSTRAINT: Mathematically impossible to double-book
```

3. Exclusion Constraint Algebra Breakdown

- unit_id WITH =: Asserts that collision checks apply only across leases for the identical physical unit.
- daterange(start_date, end_date, '[]') WITH &&: The '&&' is PostgreSQL's range overlap operator. It returns true if any single day intersects.
- WHERE (status IN ...): Excludes historical TERMINATED or EXPIRED leases, allowing future renewals.

6. ACCOUNTS RECEIVABLE: INVOICES & ITEMIZED CHARGES

AUTOMATED MONTHLY BILLING & SUBLEDGER INVARIANTS

1. Accounts Receivable Architecture

Invoices represent legal financial debts owed by a lease contract. PropLedge models invoices with parent-child line item relationships to provide granular financial accounting.

2. DDL: Invoices & Itemized Charges

// CODE LISTING: SQL DDL – ACCOUNTS RECEIVABLE SUBSYSTEM

```
-- Invoices Table (V6_create_financial_tables.sql)
CREATE TABLE invoices (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  invoice_number VARCHAR(60) NOT NULL UNIQUE,
  lease_id BIGINT NOT NULL REFERENCES leases(id) ON DELETE RESTRICT,
  invoice_date DATE NOT NULL,
  due_date DATE NOT NULL,
  total_amount NUMERIC(14,2) NOT NULL CHECK (total_amount > 0),
  balance_due NUMERIC(14,2) NOT NULL,
  status VARCHAR(30) NOT NULL DEFAULT 'PENDING'
  CHECK (status IN ('PENDING', 'PARTIALLY_PAID', 'PAID', 'OVERDUE', 'VOID')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_invoice_balance CHECK (balance_due >= 0.00 AND balance_due <= total_amount)
);

-- Itemized Line Items Table
CREATE TABLE invoice_items (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

3. Check Constraint Financial Invariants

The constraint 'CHECK (balance_due >= 0.00 AND balance_due <= total_amount)' is a mathematical invariant. An invoice can never hold a negative balance (preventing over-allocation) and cannot exceed the billed total.

7. PAYMENT ALLOCATIONS & DOUBLE-ENTRY SETTLEMENT

MULTI-INVOICE SPLIT DISTRIBUTIONS & PAYMENT VOUCHERS

1. Double-Entry Settlement Architecture

A common pitfall in novice property management projects is storing 'amount_paid' directly on the invoice. If a resident submits a single \$2,500 payment covering an overdue rent balance (\$2,000) and an electric bill (\$500), where does the payment record live? PropLedger resolves this via the 'payment_allocations' settlement junction table:

2. DDL: Payments & Payment Allocations

```
// CODE LISTING: SQL DDL – PAYMENT SETTLEMENT ARCHITECTURE

-- Payments Voucher Table
CREATE TABLE payments (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  payment_reference VARCHAR(80) NOT NULL UNIQUE,
  tenant_id BIGINT NOT NULL REFERENCES tenants(id) ON DELETE RESTRICT,
  amount NUMERIC(14,2) NOT NULL CHECK (amount > 0),
  payment_method VARCHAR(30) NOT NULL
    CHECK (payment_method IN ('ACH', 'CREDIT_CARD', 'CHECK', 'WIRE_TRANSFER')),
  status VARCHAR(30) NOT NULL DEFAULT 'SETTLED'
    CHECK (status IN ('PENDING', 'SETTLED', 'FAILED', 'REFUNDED')),
  payment_date TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Payment Allocations Junction Table (Settlement Subledger)
CREATE TABLE payment_allocations (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  payment_id BIGINT NOT NULL REFERENCES payments(id) ON DELETE RESTRICT,
```

3. Payment Allocation Accounting Rules

Rule Name	Enforcement Layer	Technical Description
Non-Negative Balance	PostgreSQL Check Constraint	balance_due >= 0.00 prevents over-applying payments.
Allocation Limit	Database Row Trigger	Sum of allocations cannot exceed payment total amount.
Lock Ordering	Spring JPA Pessimistic Lock	Invoices are locked ascending (ORDER BY id FOR UPDATE).

8. ACCOUNTS PAYABLE & FACILITIES OPERATIONS

OPERATING EXPENSES, VENDOR TRACKING & WORK ORDER COST ACCOUNTING

1. Accounts Payable (AP) & Facilities Overview

The operations subsystem tracks money leaving the property. Expenditures are classified into Operating Expenses (OpEx) for calculating Net Operating Income (NOI) and Capital Expenditures (CapEx).

2. DDL: Vendors, Expenses & Work Orders

```
// CODE LISTING: SQL DDL – ACCOUNTS PAYABLE & MAINTENANCE

-- Vendors & Contractors Table (V7__create_expenses_vendors.sql)
CREATE TABLE vendors (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  tax_id VARCHAR(50) NOT NULL, -- EIN for IRS 1099-MISC reporting
  category VARCHAR(50) NOT NULL, -- HVAC, PLUMBING, ROOFING, ELECTRICAL
  insurance_expiry_date DATE NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Operating Expenses Table
CREATE TABLE expenses (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  property_id BIGINT NOT NULL REFERENCES properties(id) ON DELETE RESTRICT,
  vendor_id BIGINT REFERENCES vendors(id) ON DELETE SET NULL,
  category VARCHAR(40) NOT NULL CHECK (category IN
    ('REPAIRS', 'UTILITIES', 'LANDSCAPING', 'INSURANCE', 'PROPERTY_TAX',
```

9. AUDIT LOGGING, JSONB DIFFS & COMPLIANCE

APPEND-ONLY IMMUTABILITY & SOX 404 REGULATORY COMPLIANCE

1. Enterprise Audit Requirements

Sarbanes-Oxley (SOX 404) and real estate trust accounting mandate an immutable audit trail of all financial and lease mutations. PropLedge implements an append-only JSONB audit log table.

2. DDL: Audit Logs Schema

```
// CODE LISTING: SQL DDL – APPEND-ONLY AUDIT TRAIL

-- Audit Logs Table (V9__create_audit_logs.sql)
CREATE TABLE audit_logs (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  table_name VARCHAR(100) NOT NULL,
  record_id BIGINT NOT NULL,
  action VARCHAR(20) NOT NULL CHECK (action IN ('INSERT', 'UPDATE', 'DELETE')),
  changed_by_user_id BIGINT REFERENCES users(id) ON DELETE SET NULL,
  old_values JSONB, -- Previous state snapshot
  new_values JSONB, -- Updated state snapshot
  client_ip VARCHAR(45),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Engine-Level Security Immutability Policy
REVOKE UPDATE, DELETE ON audit_logs FROM propledger_app;
GRANT INSERT, SELECT ON audit_logs TO propledger_app;
```

3. State Reconstruction Algebra

Because every mutation captures `old_values` and `new_values` as JSONB documents, auditors can reconstruct the exact state of any lease or invoice at any historical timestamp using standard PostgreSQL JSON operators (`->>`).

10. DATABASE MODELING INTERVIEW Q&A & SCENARIOS

SENIOR DATABASE ENGINEER & ARCHITECT INTERVIEW SCENARIOS

1. High-Yield Data Modeling Questions

Question 1: Why did you choose BIGINT over UUID for primary keys in PropLedger?

Answer: 1) Storage Footprint: BIGINT requires 8 bytes vs 16 bytes for UUID. Across millions of rows and secondary indexes, this saves gigabytes of RAM in PostgreSQL's shared_buffers. 2) B-Tree Cache Locality: Sequential BIGINT keys insert monotonically at the right leaf of the B-Tree index, preventing random I/O and page fragmentation caused by random UUIDv4 generation. 3) For external public security, we generate UUID public_ids to prevent enumeration attacks.

Question 2: How do you handle rent price escalations without breaking historical reporting?

Answer: We never update rent_amount on an existing active lease mid-term. In real estate, a price escalation requires an official Lease Amendment or Lease Renewal record. A new lease row is created linked via renewed_from_lease_id with the new rent amount and starting date. Historical invoices remain tied to their original lease contracts, preserving audit integrity.

Question 3: How do you model mixed-use commercial and residential properties?

Answer: Properties hold a property_type enum ('MIXED_USE'). The units table contains a unit_type attribute ('COMMERCIAL_OFFICE', 'RETAIL_STOREFRONT', 'APARTMENT'). Units with commercial classifications support custom lease clauses (e.g. Percentage Rent, NNN triple-net expense pass-throughs) while sharing the same underlying AR billing and maintenance infrastructure.

SENIOR CANDIDATE KEY TAKEAWAY

During database interviews, emphasize that schema design is not just about drawing boxes and arrows—it is about enforcing business invariants and mathematical truth directly in the relational engine.